CHARGEPOINT

EV Charging Station Reservation and Billing System — DBMS Course Project, Mid-Semester Design Document

Team: [Team name]
Members: [Name 1, ID] · [Name 2, ID] · [Name 3, ID] · [Name 4, ID]
GitHub: [repository URL]

1. Problem statement, users and scope

EV drivers need to find a compatible charging port, reserve a time slot, charge a vehicle and receive an accurate energy-based bill. Operators need to manage availability, tariffs, sessions, outages and revenue. ChargePoint uses PostgreSQL as the source of truth. This is an academic prototype: payments are simulated and physical chargers are not controlled.

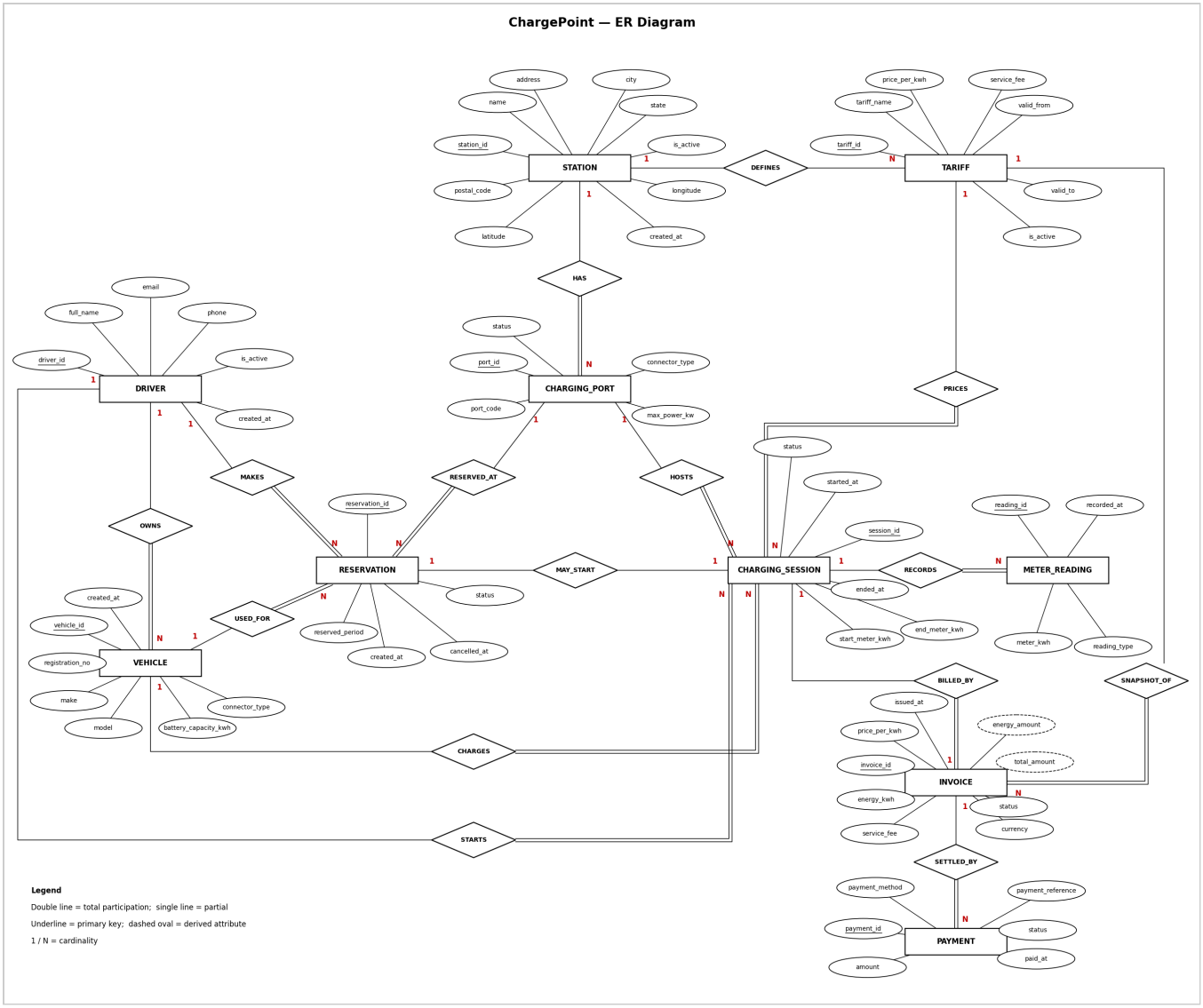
Users: Drivers manage vehicles, reservations and invoices; operators manage port status/tariffs, sessions and reports; administrators maintain reference data.

Assumptions:

- Timestamps use `TIMESTAMPTZ`; reservation ranges are half-open `[start,end)`.
- Energy = end meter − start meter; invoice = energy × price/kWh + service fee.
- Invoice price/fee are snapshots of the tariff at billing time.
- Only successful payments count toward settlement.

2. ER/EER model

Entities: DRIVERS, VEHICLES, STATIONS, CHARGING_PORTS, TARIFFS, RESERVATIONS, CHARGING_SESSIONS, METER_READINGS, INVOICES, PAYMENTS.



Notation: rectangle = entity, diamond = relationship, oval = attribute, underline = primary key, dashed oval = derived attribute, double line = total participation, single line = partial participation, 1/N = cardinality.

Relationship cardinalities

Relationship	Cardinality / business rule
Driver—Vehicle	1:N; each vehicle belongs to one driver
Station—Port / Station—Tariff	1:N for both relationships
Driver, Vehicle, Port—Reservation	Each reservation references one of each
Reservation—Session	Reservation-linked session is optional and at most one; walk-in sessions allowed
Session—Meter Reading	1:N; readings are timestamped
Session—Invoice	1:0..1; at most one invoice per session
Invoice—Payment	1:N; supports partial payments and failed attempts

Participation constraints

- **Total participation:** every vehicle has a driver; every port belongs to a station; every reservation has a driver, vehicle and port; every session has a driver, vehicle, port and tariff; every meter reading, invoice and payment has its parent.
- **Partial participation:** a reservation may never become a session (cancelled/no-show); a session may not yet have an invoice; a tariff may be global (no station).

3. Relational schema

Relation	Attributes (type)	Keys
drivers	driver_id BIGINT, full_name VARCHAR(120), email VARCHAR(254), phone VARCHAR(25), created_at TIMESTAMPTZ, is_active BOOLEAN	PK driver_id; UNIQUE email, phone
vehicles	vehicle_id BIGINT, driver_id BIGINT, registration_no VARCHAR(20), make, model VARCHAR(60), battery_capacity_kwh NUMERIC(7,2), connector_type VARCHAR(20)	PK vehicle_id; FK driver_id; UNIQUE registration_no
stations	station_id BIGINT, name, address, city, state, postal_code VARCHAR, latitude/longitude NUMERIC(9,6), is_active BOOLEAN	PK station_id
charging_ports	port_id BIGINT, station_id BIGINT, port_code VARCHAR(20), connector_type VARCHAR(20), max_power_kw NUMERIC(7,2), status VARCHAR(20)	PK port_id; FK station_id; UNIQUE(station_id, port_code)
tariffs	tariff_id BIGINT, station_id BIGINT, tariff_name VARCHAR(100), price_per_kwh NUMERIC(10,4), service_fee NUMERIC(10,2), valid_from/valid_to TIMESTAMPTZ, is_active BOOLEAN	PK tariff_id; FK station_id (nullable); UNIQUE(station_id, tariff_name, valid_from)
reservations	reservation_id BIGINT, driver_id, vehicle_id, port_id BIGINT, reserved_period TSTZRANGE, status VARCHAR(20), created_at, cancelled_at TIMESTAMPTZ	PK reservation_id; FK driver_id, vehicle_id, port_id
charging_sessions	session_id BIGINT, reservation_id, driver_id, vehicle_id, port_id, tariff_id BIGINT, started_at/ended_at TIMESTAMPTZ, start_meter_kwh/end_meter_kwh NUMERIC(12,3), status VARCHAR(20)	PK session_id; FK reservation_id (UNIQUE, nullable), driver_id, vehicle_id, port_id, tariff_id
meter_readings	reading_id BIGINT, session_id BIGINT, recorded_at TIMESTAMPTZ, meter_kwh NUMERIC(12,3), reading_type VARCHAR(20)	PK reading_id; FK session_id
invoices	invoice_id BIGINT, session_id, tariff_id BIGINT, energy_kwh NUMERIC(12,3), price_per_kwh NUMERIC(10,4), service_fee NUMERIC(10,2), energy_amount/total_amount NUMERIC(12,2) (generated), currency CHAR(3), issued_at TIMESTAMPTZ, status VARCHAR(20)	PK invoice_id; FK session_id (UNIQUE), tariff_id
payments	payment_id BIGINT, invoice_id BIGINT, amount NUMERIC(12,2), payment_method VARCHAR(20), payment_reference VARCHAR(100), status VARCHAR(20), paid_at TIMESTAMPTZ	PK payment_id; FK invoice_id; UNIQUE payment_reference

Candidate keys: drivers.email, drivers.phone; vehicles.registration_no; charging_ports(station_id, port_code); charging_sessions.reservation_id; invoices.session_id; payments.payment_reference.

4. Integrity constraints

- **PK / FK / UNIQUE / NOT NULL** on all relations as listed above; foreign keys use `ON DELETE RESTRICT` (meter readings cascade with their session).
- **CHECK:** battery capacity and port power > 0; prices, fees, energy and meter values ≥ 0; payment amount > 0; coordinates in valid range; enumerated values for connector type, port status, reservation/session/invoice/payment status and payment method; `valid_to > valid_from`; `ended_at ≥ started_at`; `end_meter_kwh ≥ start_meter_kwh`; session status consistent with end time/meter.
- **Exclusion constraint:** GiST exclusion on `(port_id, reserved_period)` rejects overlapping active reservations for the same port.
- **Partial unique index:** at most one in-progress session per port.
- **Trigger:** recalculates invoice status (unpaid / partially paid / paid) from successful payments.

5. Normalization and design justification

1NF: atomic fields; multiple readings, payments, vehicles and ports are separate rows. **2NF:** single-column primary keys; non-key attributes depend on the key. **3NF:** driver, station, port, tariff, session, invoice and payment facts live in separate relations; names/contact/address are not copied into reservation rows. Invoice price and fee are intentionally snapshotted so tariff changes do not rewrite historic bills.

Reservation overlap is enforced in PostgreSQL rather than only by a pre-check in the application, because concurrent requests can race. The range constraint is the final guard.

6. Application design

Architecture and technology stack

Frontend: responsive HTML/CSS. Backend: Flask/Python. Database access: psycopg parameterized SQL. Database: PostgreSQL 15+. Tests: pytest. Payments and charger control are simulated/not integrated.

```
Browser UI  <-->  Flask routes  <-->  psycopg SQL  <-->  PostgreSQL
                                                        | constraints / transactions
                                                        | trigger: payment status
                                                        | view: station utilization
                                                        ` indexes / reporting queries
```

End-to-end workflows

- **A — Reservation:** driver chooses station/port/time; backend validates input and inserts reservation; PostgreSQL rejects overlaps; UI reports confirmation/conflict.
- **B — Session and invoice:** operator records start meter and tariff; later records end meter/time; a transaction closes the session and inserts invoice; end reading must be ≥ start reading.
- **C — Payment:** driver records a simulated payment; trigger recalculates invoice state as unpaid, partially paid or paid from successful payments.
- **D — Reporting:** operator reviews utilization, energy sold, revenue, outstanding balances and reservation issue rates using joins, aggregates, CTEs and a view.

7. SQL and DBMS features

Reporting queries cover station utilization, monthly revenue, port usage, driver spend, cancellation/no-show rate, CTE-based session duration, outstanding invoices, connector compatibility using NOT EXISTS, upcoming reservations, and tariff comparison. DBMS features include transactions, exclusion constraint, trigger, view, indexes and integrity constraints.

8. Progress and team contributions

Completed: ER design, PostgreSQL schema with constraints, indexes, view and trigger; 10 reporting queries; transaction scripts for reservation, session completion/invoicing and payment; synthetic data generator (500+ rows); Flask application with station overview and reservation creation; tests.

Next: session/invoice, payment and reporting screens in the application.

Member (Name, ID)	Contribution
[Member 1]	[contribution]
[Member 2]	[contribution]
[Member 3]	[contribution]
[Member 4]	[contribution]